
Intelligence Engineering

Solving Real World Problems At Scale

Prof. Dr.-Ing. Mark Schutera | September 10, 2026 · *drifting stream*

LECTURE _

Orientation

what this course is, what it assumes, what you are going to know after.

You have been given a lot of raw **data**,

**and you want to engineer the shit
out of it.**

You have been given a business problem,

**and you want to choose the right
metrics and objectives to solve it.**

Your initial models perform well in offline experiments,

and you want to deploy them at scale.

You want to harden your deployments

**to quickly detect, debug, and
address issues that arise in
production.**

There is a lot of manual work and handling involved so far,

**and you want to operation vacation
your machine learning product or
service.**

You want to learn about a framework and foundation for building machine learning systems

**that can be shared and reused
across machine learning use cases
and data products.**

1

Overview

Machine Learning and Data Engineering

The lecture in one slide

Lecture	Contents
I	Overview of Machine Learning Systems
II	Introduction to Machine Learning Systems Design
III	Data Engineering Fundamentals
IV	Training Data
V	Feature Engineering
VI	Model Development and Offline Evaluation
VII	Model Deployment and Prediction Service
VIII	Data Distribution Shifts and Monitoring
IX	Continual Learning and Test in Production
X	Infrastructure and Tooling for MLOps
XI	The Human Side of Machine Learning

The lecture in one slide

I Overview of Machine Learning Systems

When to Use Machine Learning

Understanding Machine Learning Systems

II Introduction to Machine Learning Systems Design

Business and ML Objectives

Requirements for ML Systems

Iterative Process

Framing ML Problems

Mind Versus Data

III Data Engineering Fundamentals

Data Sources

Data Formats

Data Models

Data Storage Engines and Processing

Modes of Dataflow

Batch Processing Versus Stream Processing

The lecture in one slide

$P(y|x)$

IV Training Data

Sampling

Labeling

Class Imbalance

Data Augmentation

V Feature Engineering

Learned Features Versus Engineered Features

Common Operations

Data Leakage

Engineering Good Features

VI Model Development and Offline Evaluation

Model Development and Training

Model Offline Evaluation

[1]

The lecture in one slide

$P(y|x)$

VII Model Deployment and Prediction Service

ML Deployment Myths
Batch Prediction Versus Online Prediction
Model Compression
ML on the Cloud and on the Edge

VIII Data Distribution Shifts and Monitoring

Causes of ML System Failures
Data Distribution Shifts
Monitoring and Observability

IX Continual Learning and Test in Production

Continual Learning
Test in Production

[1]

The lecture in one slide

X Infrastructure and Tooling for MLOps

Storage and Compute
Development Environment
Resource Management
ML Platform
Build Versus Buy

XI The Human Side of Machine Learning

User Experience
Team Structure
Responsible AI

Course Work

Presentation (15 min) + Q&A (open)

You get recordings of a bee hive entrance, and you are to contribute by data engineering, an annotation policy, a prediction service or specialized tooling.

Open whiteboard, three slides or one page only. Be ready when the class starts.

The bee



Course Work

What I am looking for ..

Introduction by Motivation and Literature

Method by Explanation and or Demonstration

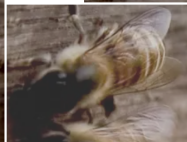
Results show what was achieved and how well

Discussion on findings, limitations and what's next

Smart and Aesthetic Thoughts, Designs, Ideas and **Questions**.

The bee

bee 1 | 0.97



Extra Mile

Special projects

If you are in for an extra mile, let me know. I have special projects that I can assign to you.

Wrapped in the same presentation and Q&A format, yes it will be about Bananas, and yes it will be more challenging, more time and effort.

Classic loose loose short term, yet **win long term.**

A large banana





By the end of this lecture,

**Get yourself a presentation slot
and brainstorm a project idea to
work on.**

References

- 1 Chip Huyen. *Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications*. Sebastopol, CA: O'Reilly Media, 2022.

LECTURE I

Overview of a Machine Learning System

1

The small part and the big picture

Don't confuse the model with the machine learning system.

Enabling the delivery of the logic

The **model** is the small part of a machine learning system. With foundation models and AI engineering, that part got smaller still. The essential work of a machine learning system remains and stacks around business understanding, infrastructure, design pattern and data engineering.

Before learning about solving a problem with machine learning, let's first assess when it is actually a good idea to do so.

(1) Learn

The problem you are solving, requires your system to have the capability to learn. Learning can look different depending on the problem and its context and boundary conditions. You already know about supervised learning, unsupervised learning and reinforcement learning paradigms.

Machine learning is an approach to (1) learn (2) complex patterns from (3) existing data and use these patterns to make (4) predictions on (5) unseen data.

(2) Pattern

Learning requires the existence of a pattern, as opposed to noise or randomness. It is not trivial nor obvious whether there is a pattern in the data, and whether it is learnable. Complex pattern is more difficult to embed or state explicitly, this is where machine learning has an edge over classical approaches.

Machine learning is an approach to (1) learn (2) complex patterns from (3) existing data and use these patterns to make (4) predictions on (5) unseen data.

(3) Data

Data is available, accessible or acquirable. The data is of sufficient quality and quantity to support learning, (and to validate for that matter). If your approach does not require data, it is because you build on embeddings or foundation models, which themselves are built on data in the first place, or you need to rely on data acquisition and curation as part of your system.

Machine learning is an approach to (1) learn (2) complex patterns from (3) existing data and use these patterns to make (4) predictions on (5) unseen data.

(4) Predictions

Machine learning is about making predictions, always. Predictions can be of different types, such as classification, regression, ranking, clustering, generation and on and on. By reframing problems in terms of predictions, we can leverage machine learning to solve them. Predictions are approximations by nature and therefore come with uncertainty, which needs to be accounted for.

Machine learning is an approach to (1) learn (2) complex patterns from (3) existing data and use these patterns to make (4) predictions on (5) unseen data.

(5) Unseen Data

In a closed system or domain, the data is finite and the system can be fully specified by optimization. However, often the domain is open and infinite, and the system needs to be able to generalize to unseen data. Of course, the unseen data is not completely unknown, it is drawn from the same distribution (i.i.d.) as the training data and is thus able to generalize to the unseen data inside a specific domain or distribution.

Machine learning is an approach to (1) learn (2) complex patterns from (3) existing data and use these patterns to make (4) predictions on (5) unseen data.

Catalysts

A catalyst is a substance that increases the rate of reaction without undergoing change itself.

The task is repetitive.

In a sense that we have a lot of data to learn a pattern from.

It scales.

The need to scale a specific prediction capability, not only a few times.

The context changes.

Meaning that hard coded or other solutions will be brittle and break.

Being wrong is not too bad.

Approximations on unseen data. Failures need to be manageable.

When Not to Use Machine Learning

It is unethical.

Well, that might also be a general rule. But doing stuff at scale, without human oversight, has its implications.*

There's a simple solution.

A simple solution, heuristic, or baseline should always be the first step regardless.

It's not economically viable.

The value of the prediction, needs to be greater than the cost of the system, including development, deployment and maintenance.

* More? Enroll to the Philosophy of AI course at ZDI.

Describe two use-cases and the rationale for whether or not to use machine learning.

One Machine Learning Use-Case
and **One** which is not.

2

Understanding Machine Learning Systems

Research and Production.

Horizontal versus Vertical

$P(y|x)$

Research	Production
Requirements	
Computational priority	
Data	
Fairness	
Interpretability	

[1]

Different stakeholders and requirements

$P(y|x)$

A mobile app recommends restaurants and charges them a **10% service fee** per order, so expensive orders pay the app more.

ML engineers

Want a model that recommends the restaurants users will most likely order from, via more data and a more complex model.

Sales team

Wants a model that recommends the more expensive restaurants, since these bring in more service fees.

Product team

Every increase in latency leads to a drop in orders, so wants the recommendations in less than **100 milliseconds**.

ML platform team

Woken up in the middle of the night by scaling problems, so wants to hold off on model updates and improve the ML platform.

Manager

Wants to maximize the margin, and one way to achieve this might be to let go of the ML team.

Clicks and money are two different objectives. Spoiler: one model for each objective, then combine their predictions.

[1]

Must-have or nice-to-have

Model A recommends the restaurants that users are most likely to click on, model B the restaurants that will bring in the most money for the app. Neither returns its recommendations in less than **100 milliseconds**. Which model should be deployed to the users?

- **Must-have.** The company finds that if the model takes over 100 milliseconds, 10% of users would lose patience and close the app. Then neither model A nor model B will work.
- **Nice-to-have.** You might still want to consider model A or model B.

If a simple model can do a reasonable job, complex models must perform significantly better to justify the complexity.

Ensembling is popular among the winners of many ML competitions, including the famed \$1 million Netflix Prize, and yet it is not widely used in production: too complex to be useful, slower to make predictions, harder to interpret the results.

Computational priorities

Research: fast training

- Training is the bottleneck: many models, each doing multiple passes over the data.
- Prioritizes **high throughput**, how many queries are processed in a period of time.

Production: fast inference

- Once deployed, inference is the bottleneck: the model's job is to generate predictions.
- Prioritizes **low latency**, the time from receiving a query to returning the result.

Processing	Average latency	Throughput
One query at a time	10 ms	100 queries/second
One query at a time	100 ms	10 queries/second
Batches of 10 queries	10 ms	1,000 queries/second
Batches of 50 queries	20 ms	2,500 queries/second

Batched, higher latency might also mean higher throughput.

Latency is a distribution

Ten requests, latency in ms: 100, 102, 100, 100, 99, 104, 110, 90, **3,000**, 95.

The average latency is 390 ms, which makes your system seem slower than it actually is.

Better to think in percentiles: p50 is the median, p90 and p99 expose the outliers.

The p90 here is 3,000 ms, the one request to investigate.

Latency matters a lot.

2017, Akamai: a 100 ms delay can hurt conversion rates by 7%.

2019, Booking.com: about 30% more latency cost about 0.5% in conversion rates, “a relevant cost for our business”.

2016, Google: more than half of mobile users leave a page that takes more than three seconds to load.

On Amazon the slowest requests come from the most valuable customers.

Research: static

- Clean and well formatted
- Static, so the community can benchmark
- Quirks of the dataset are known
- Open source scripts to feed your models
- Historical data, already stored somewhere

Production: constantly shifting

- Messy, noisy, possibly unstructured
- Likely biased, and you do not know how
- Labels sparse, imbalanced, or incorrect
- Changing requirements can mean relabeling
- Privacy and regulatory concerns
- Generated by users, systems, third parties

Andrej Karpathy, director of AI at Tesla, drew the data problems of his PhD against those of his time at Tesla.

[1]

Fairness

During the research phase, a model is not yet used on people, so it is easy for researchers to put off fairness as an afterthought.

“Let’s try to get state of the art first and worry about fairness when we get to production.”

When it gets to production, it is too late.

Optimize for better accuracy or lower latency and you can show that you beat state of the art.

As of writing the book, there is no equivalent state of the art for fairness metrics.

Fairness at scale

Machine learning algorithms do not predict the future, they encode the past, thus perpetuating the biases in the data.

- Your loan application might be rejected because the algorithm picks on your **zip code**, which embodies biases about one's socioeconomic background.
- Your resume might be ranked lower because the ranking system employers use picks on the **spelling of your name**.
- Your mortgage might get a higher interest rate because it relies partially on **credit scores**, which favor the rich and punish the poor.

Deployed at scale, they discriminate at scale, judging millions in split seconds.

In 2019, Berkeley researchers found that both face-to-face and online lenders rejected **1.3 million** creditworthy Black and Latino applicants between 2008 and 2015. With the income and credit scores kept and the race identifiers deleted, the mortgage application was accepted.

McKinsey & Company, 2019: only **13%** of the large companies surveyed said they are taking steps to mitigate risks to equity and fairness.

Interpretability

In early 2020, Turing Award winner Geoffrey Hinton posed a heatedly debated question. Suppose you have cancer and you have to choose.

Black box AI surgeon

90% cure rate, and it cannot explain how it works.

Human surgeon

80% cure rate.

“Do you want the AI surgeon to be illegal?”

Put to a group of 30 technology executives at public nontech companies, half of them wanted the AI surgeon to operate on them. The other half wanted the human surgeon.

Interpretability in production

Most machine learning research is still evaluated on a single objective, model performance, so researchers are not incentivized to work on model interpretability. In industry, interpretability is not just optional for most use cases but a requirement.

- Important for **users**, both business leaders and end users, to understand why a decision is made, so that they can trust a model and detect potential biases.
- Important for **developers**, to be able to debug and improve a model.

Most of us are comfortable using a microwave without understanding how it works. Many do not feel the same way about AI, especially if that AI makes important decisions about their lives.

As of 2019, only **19%** of large companies are working to improve the explainability of their algorithms.

Discussion

$P(y|x)$

[1]

Machine Learning Systems Versus Traditional Software

$P(y|x)$

Traditional software engineering

- An underlying assumption that **code and data are separated**.
- Keep things as modular and separate as possible: separation of concerns.
- Many of its tools do work for machine learning.

Machine learning systems

- **Part code, part data**, and part artifacts created from the two.
- Applications with the most and best data win: companies improve data, not algorithms.
- Data can change quickly, so applications need to be adaptive.

ML production would be a much better place if ML experts were better software engineers.

[1]

Test and version the data too

In traditional software engineering, you only need to focus on testing and versioning your code. With machine learning, we have to test and version our data too, and **that is the hard part.**

- How to version large datasets?
- How to know if a data sample is good or bad for your system?

Not all data samples are equal. Some are more valuable to your model than others.

Indiscriminately accepting all available data might hurt performance and make your model susceptible to data poisoning attacks.

Your model has already trained on **one million** scans of normal lungs and only **one thousand** scans of cancerous lungs.

A scan of a cancerous lung is much more valuable than a scan of a normal lung.

Size, speed and visibility

Size

As of 2022 it is common for models to have **hundreds of millions, if not billions, of parameters**, needing gigabytes of RAM.

Speed

Getting large models into production, **especially on edge devices**, is a massive challenge, as is running them fast enough.

Visibility

Monitoring and debugging in production is also nontrivial: models get complex, with a **lack of visibility** into their work.

An autocompletion model is useless if the time it takes to suggest the next character is longer than the time it takes for you to type.

The good news: these challenges are being tackled at a breakneck pace

In 2018 people said BERT was too big, too complex and too slow to be practical. The pretrained large BERT model has **340 million** parameters and is **1.35 GB**. Two years later, BERT and its variants were used in almost every English search on Google.

References

- 1 Chip Huyen. *Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications*. Sebastopol, CA: O'Reilly Media, 2022.

This is the last slide.
Look how far you've come.